

Thesis progress report

LLM-Guided Neural Network Compilation for Real-time UAV Edge Inference

Atul Kumar Gupta · 25MCSS06

The aircraft only loads the file

Measurement ends with an ONNX file and a sidecar of session options. This report is the program that loads that file on the simulated aircraft.

Gazebo publishes the camera on the ROS topic `/nnc/camera/image_raw`. The simulated airframe is `nnc_x500_cam`. PX4 SITL flies beside it. The ROS node `inference_node` loads the artifact and runs it on those frames.

The node does not send flight setpoints. The compiler does not read the flight controller. The aircraft is a place to run the file.

The sidecar next to the ONNX file stores the plan and the session options, so the node uses the same thread count that was measured. Camera frames can be written to disk with `record_frames.py`. That recording is a separate experiment from the compile table.

The early write-up treated the flying stack as the thesis, so the compiler was going to talk to PX4. That would have made a bad setpoint look like a bad compile. The node was therefore forbidden from publishing setpoints, and the compiler was forbidden from reading them. Getting a picture out of Gazebo was its own failure: one render path crashed, and the camera topic was empty until the bridge and the airframe matched. Those are environment facts. They are not compile results, and they are not written down as if they were.

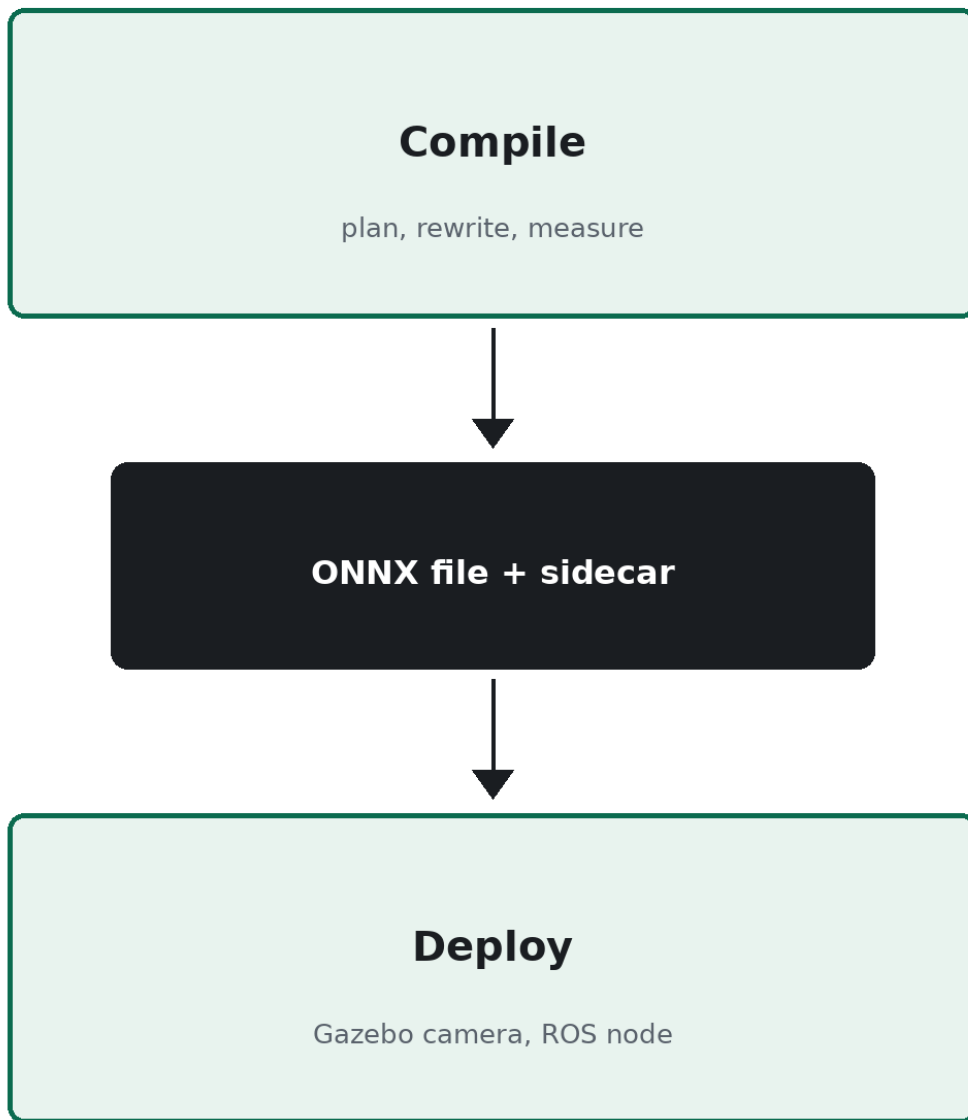


Figure 1. The compile side and the aircraft share only the artifact.

The simulated aircraft only runs the file. It does not choose the recipe. A remote model can still be the thing that names the plan.